

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
CENTRO UNIVERSITARIO DE MAZATENANGO, SUCHITEPÉQUEZ
INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA
COMPUTACIÓN OCTAVO SEMESTRE SECCIÓN "A" ANALISIS DE SISTEMAS II
ING. ANGEL ATILIO MALTEZ CIFUENTES

NINCY ABIGAIL RODRIGUEZ CHÁVEZ

3090-23-5765



Fase de Diseño

FECHA: 27 / 09 / 2026

Contenido

1. Objetivo del documento
2. Planificación de trabajo
 - 2.2 Diagrama de Gantt
3. Arquitectura de software
4. Documentación de componentes
5. Diseño de experiencia del usuario (UX/UI)
6. Interfaces frontend y backend
7. Prototipo
8. Estrategia de despliegue público
9. Riesgos, control y seguimiento
10. Criterios de cierre de la fase de diseño

1. Objetivo del documento

Este documento define la planificación realista, la arquitectura de software, los componentes, la experiencia del usuario y el prototipo del Sistema para Administración del Asilo de Ancianos Cabeza de Algodón. Su propósito es servir como guía de diseño y ejecución para completar el sistema antes de la entrega final, manteniendo coherencia entre requisitos, desarrollo, pruebas y documentación.

Enfoque adoptado: desarrollo local en Windows con Visual Studio y SQL Server; publicación final mediante infraestructura en la nube, evitando exponer directamente la red doméstica.

2. Planificación de trabajo

La planificación se organiza por fases y considera el estado real del proyecto: análisis y diseño completados, base técnica y módulos iniciales desarrollados, y módulos clínicos/financieros pendientes. Se propone una dedicación promedio de 2 horas por día entre semana y bloques adicionales de fin de semana, ajustables según carga académica.

Fase	Periodo	Horas estimadas	Resultado
Análisis	03/08/2026 - 07/08/2026	30 h	ERS, UML, casos de uso, ERD y factibilidad
Diseño	10/08/2026 - 28/08/2026	33 h	Arquitectura, componentes, UX, prototipo y SQA
Desarrollo base	29/08/2026 - 23/09/2026	44 h	Solución, base de datos, autenticación, internos, notificaciones y familiares
Desarrollo funcional	24/09/2026 - 10/10/2026	48 h	Ficha médica, solicitudes, visitas, laboratorio, farmacia, finanzas y reportes
Seguridad y pruebas	11/10/2026 - 14/10/2026	17 h	Hash de contraseñas, auditoría, integración y SQA
Implementación final	15/10/2026 - 16/10/2026	9 h	Manual, despliegue, revisión y entrega

2.1 Hitos principales

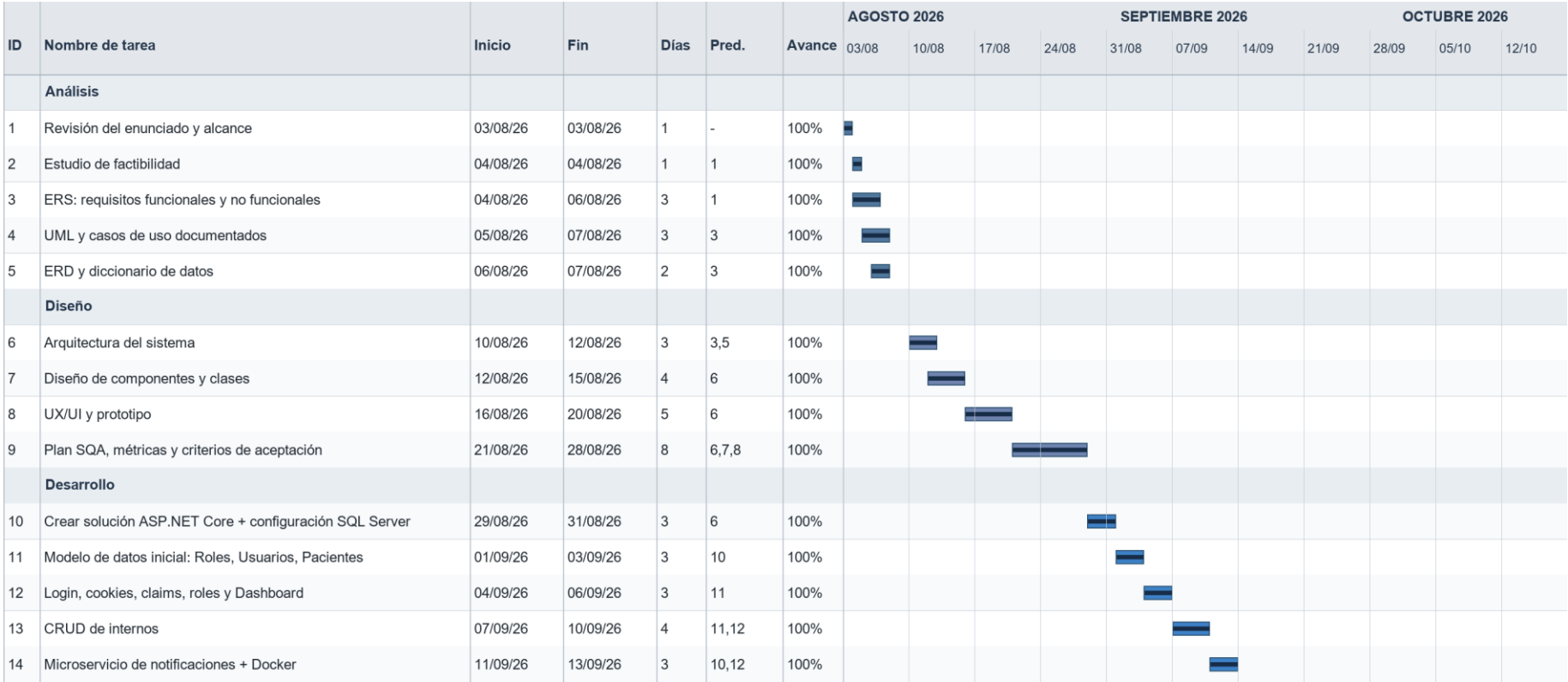
Hito	Fecha objetivo	Criterio
Análisis terminado	07/08/2026	ERS, UML y ERD documentados
Diseño terminado	28/08/2026	Arquitectura, componentes, UX y prototipo documentados
Base funcional	18/09/2026	Login, roles, Dashboard, internos y notificaciones funcionando
Módulos clínicos	07/10/2026	Ficha, solicitudes, visita, laboratorio y farmacia integrados
Sistema integrado	14/10/2026	Finanzas, reportes, seguridad y pruebas completadas
Entrega final	16/10/2026	Sistema, documentación, manual y repositorio listos

El detalle editable de actividades, dependencias, horas, porcentaje de avance, recursos e hitos se conserva en el archivo Planificacion_Proyecto_Asilo_Cabeza_de_Algodon.xlsx. ProjectLibre se utilizará para gestionar y dar seguimiento al cronograma.

2.2 Diagrama de Gantt

Análisis, diseño y desarrollo base

Periodo del proyecto: 03/08/2026 al 16/10/2026. Escala semanal con fechas de inicio de semana.



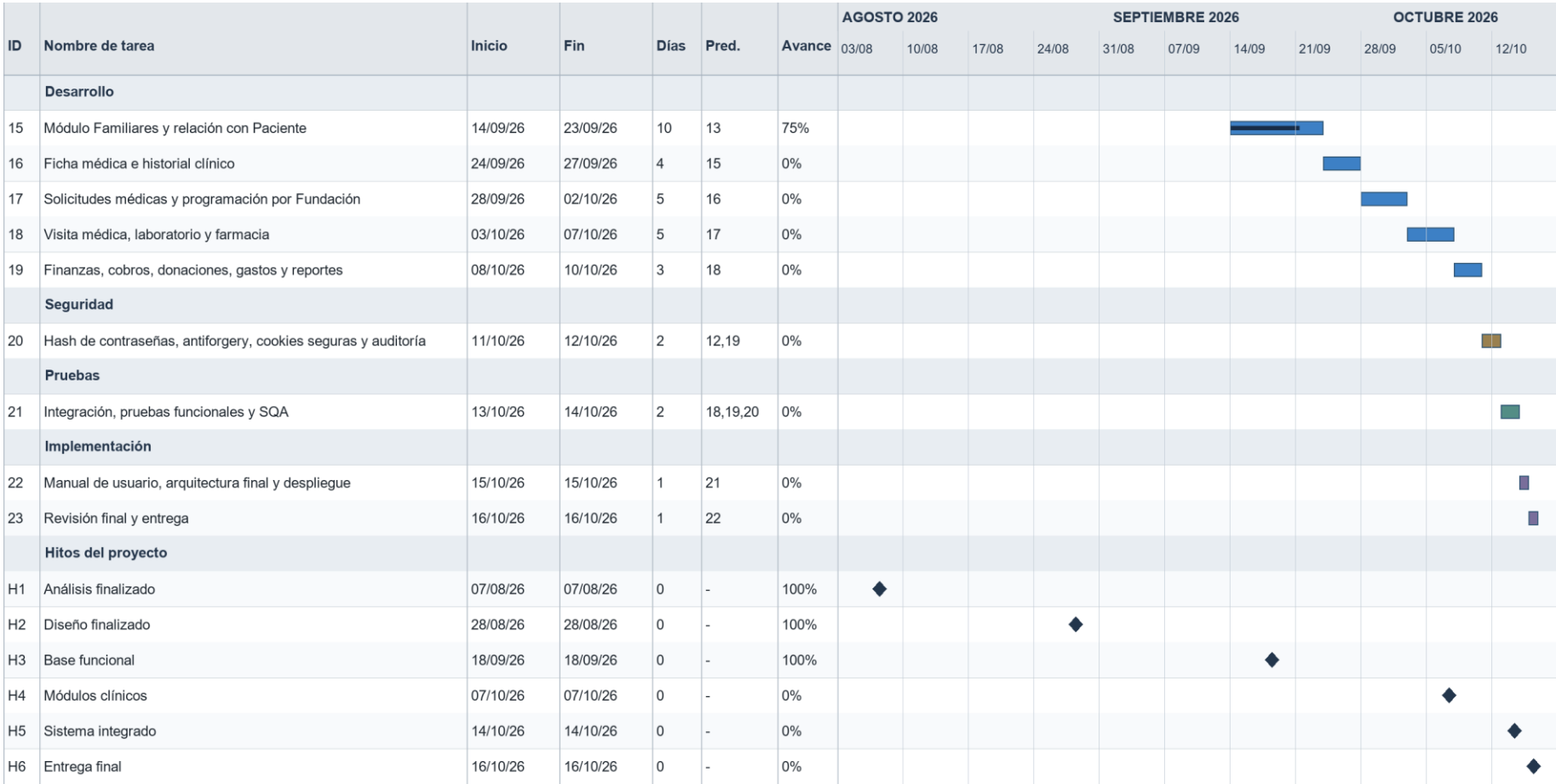
Barras de color: duración planificada. Trazo oscuro: avance registrado. Rombo: hito de duración cero. Pred.: ID de las actividades predecesoras.

La duración se expresa en días calendario, incluidos los fines de semana, según el Excel. Las dependencias se muestran como referencias; su tipo de vínculo no está definido en la planificación y existen actividades superpuestas. Al configurarlas en ProjectLibre, se deben validar los vínculos y el calendario para conservar estas fechas.

2.2 Diagrama de Gantt continuación

Desarrollo funcional, cierre e hitos del proyecto

Periodo del proyecto: 03/08/2026 al 16/10/2026. Escala semanal con fechas de inicio de semana.



Barras de color: duración planificada. Trazo oscuro: avance registrado. Rombo: hito de duración cero. Pred.: ID de las actividades predecesoras.

Fuente: hojas Cronograma e Hitos del archivo de planificación. Se conserva el avance registrado: 14 tareas completadas, Familiares al 75 % y las demás pendientes. Esfuerzo total: 181 horas; avance ponderado por horas: 58,0 %. Responsable principal: Desarrolladora.

3. Arquitectura de software

Se adopta una arquitectura web por capas con ASP.NET Core MVC. La separación de responsabilidades permite que la interfaz, la lógica de negocio, el acceso a datos y los servicios externos evolucionen de forma independiente.

3.1 Arquitectura lógica

CAPA DE PRESENTACIÓN Razor Views + HTML5 + CSS3 + Bootstrap + JavaScript Login, Dashboard, formularios, tablas y reportes
CAPA WEB / CONTROLADORES ASP.NET Core MVC Controllers + ViewModels Validación de solicitudes, autorización y coordinación de casos de uso
CAPA DE APLICACIÓN Y NEGOCIO Servicios del dominio Reglas médicas, financieras, notificaciones, permisos y auditoría
CAPA DE DATOS Entity Framework Core + AsiloDbContext Entidades, relaciones, consultas y migraciones
INFRAESTRUCTURA SQL Server + API de Notificaciones + Docker + servicio SMTP + Git/GitHub

3.2 Flujo de una solicitud típica

1. El usuario accede desde un navegador mediante HTTPS.
2. ASP.NET Core valida la cookie de autenticación y los roles del usuario.
3. El controlador recibe la petición y valida el ViewModel.
4. El servicio correspondiente aplica reglas de negocio.
5. Entity Framework Core consulta o modifica SQL Server.
6. Cuando se requiere una notificación, Asilo.Web invoca Asilo.Notificaciones.Api.
7. La respuesta vuelve al controlador y se presenta al usuario mediante una vista Razor.

3.3 Arquitectura de despliegue

Nodo / servicio	Tecnología	Responsabilidad
Cliente	Chrome / Edge / Firefox	Acceso al sistema
Servidor web	Contenedor ASP.NET Core MVC	Ejecuta Asilo.Web
Microservicio	Contenedor ASP.NET Core Web API	Procesa notificaciones
Base de datos	SQL Server	Persistencia de información
Correo	SMTP o proveedor equivalente	Entrega de correos al familiar
Repositorio	GitHub	Control de versiones y respaldo del código

Decisión de despliegue: para el acceso público final se recomienda una VPS o servicio de nube con HTTPS. El desarrollo puede continuar en la computadora local. SQL Server LocalDB se utiliza solo en desarrollo; para contenedores o nube se debe usar una instancia accesible de SQL Server.

4. Documentación de componentes de software

Los componentes se diseñan alrededor de las responsabilidades funcionales del sistema. Cada componente agrupa clases, controladores, servicios y entidades relacionadas.

Componente	Clases principales	Responsabilidad	Dependencias
Autenticación y autorización	LoginController, LoginViewModel, Usuario, Rol	Iniciar/cerrar sesión, crear Claims, validar roles, acceso denegado	AsiloDbContext
Usuarios y roles	Usuario, Rol, futuro UsuariosController	Administrar cuentas, estados y roles	Autenticación
Internos	Anciano, AncianosController	CRUD de pacientes internos	Familiares, ficha médica
Familiares	Familiar, FamiliaresController	CRUD y relación Paciente-Familiar	Internos
Ficha médica	FichaMedica, HistorialMedico, FichaMedicaController	Padecimientos, antecedentes, observaciones e historial	Internos, visitas
Solicitudes médicas	SolicitudMedica, SolicitudesController	Referencia, especialidad, enfermero, estado y programación	Médicos, fundación, notificaciones
Visitas médicas	VisitaMedica, VisitasController	Diagnóstico, tratamiento, observaciones	Solicitud, laboratorio, farmacia
Laboratorio	ExamenLaboratorio, ResultadoLaboratorio	Exámenes, resultados, costos y estado	Visitas
Farmacia	Medicamento, Receta	Medicamento indicado, entrega y costos	Visitas
Finanzas	Cobro, Pago, Donacion, Gasto	SalDOS, cobros, pagos, donaciones y egresos	Visitas, laboratorio, farmacia
Reportes	ReportesController / servicio	Reportes médicos, administrativos y financieros	Todos los módulos
Notificaciones	NotificacionesClient, Asilo.Notificaciones.Api	Procesar avisos al familiar	Solicitudes médicas
Persistencia	AsiloDbContext, migraciones EF Core	Acceso y persistencia de datos	SQL Server
Auditoría	Bitacora / servicio de auditoría	Registrar cambios sensibles	Usuarios y módulos críticos

4.1 Diseño basado en clases

Las clases de dominio representan las entidades del ERD, mientras que los controladores coordinan los casos de uso. Los servicios concentran lógica que no debe permanecer dentro de las vistas ni de los controladores.

Tipo de clase	Ejemplos	Uso
Entidad	Anciano, Familiar, Usuario, Rol, SolicitudMedica	Representan datos persistentes
ViewModel	LoginViewModel y futuros formularios específicos	Reciben y validan datos de la interfaz
Controller	LoginController, AncianosController, FamiliaresController	Gestionan solicitudes HTTP y navegación
Service	NotificacionesClient y servicios futuros	Lógica reutilizable e integración externa
DbContext	AsiloDbContext	Unidad de acceso a SQL Server con EF Core

5. Diseño de experiencia del usuario (UX/UI)

El sistema será utilizado por perfiles con funciones muy diferentes; por ello se prioriza una interfaz clara, consistente y centrada en tareas. El usuario debe ver únicamente aquello que necesita para su rol.

5.1 Patrones de diseño de interfaz

Patrón	Aplicación en el sistema
Dashboard por rol	Muestra solo módulos autorizados para Administrador, Médico, Laboratorio, Farmacia, etc.
CRUD consistente	Listar → Registrar → Editar → Confirmar eliminación → Retroalimentación
Divulgación progresiva	Cada pantalla muestra primero información esencial; detalles clínicos o financieros se abren cuando se necesitan
Validación en contexto	Mensajes junto al campo y validación antes de guardar
Confirmación de acciones críticas	Eliminar registros o cerrar procesos clínicos/financieros requiere confirmación
Retroalimentación inmediata	Mensajes de registro exitoso, acceso denegado, errores y estados
Navegación predecible	Dashboard como punto central y botones Volver/Cancelar
Diseño responsivo	Bootstrap para adaptarse a diferentes tamaños de pantalla
Control por rol	La interfaz oculta acciones no autorizadas además de protegerlas en backend

5.2 Principios UX

- Consistencia: mismos estilos de botones, formularios y tablas en todos los módulos.
- Prevención de errores: listas desplegables para roles, pacientes, sexo, estados y categorías.
- Carga cognitiva baja: formularios divididos por secciones relacionadas.
- Visibilidad del estado: mostrar si una solicitud está pendiente, programada, atendida o cerrada.
- Accesibilidad básica: etiquetas visibles, contraste adecuado, navegación con teclado y mensajes comprensibles.
- Seguridad usable: acceso denegado claro, cierre de sesión visible y opciones limitadas por rol.

6. Clases de interfaces del frontend y backend

6.1 Frontend

Interfaz / vista	Clase o modelo asociado	Función
Login	LoginViewModel	Captura usuario/correo y contraseña
Dashboard	Claims/Roles	Presenta módulos permitidos por rol
Internos - Index	IEnumerable<Anciano>	Lista internos registrados
Internos - Registrar/Editar	Anciano	Captura y valida datos del interno
Familiares - Index	IEnumerable<Familiar>	Lista familiares y paciente relacionado
Familiares - Registrar	Familiar	Registra familiar y selecciona paciente
Solicitudes	SolicitudMedica	Crear, consultar y dar seguimiento
Laboratorio	ExamenLaboratorio/Resultado	Registrar resultados y costos
Farmacia	Receta/Medicamento	Registrar entrega e información de medicamento
Reportes	ViewModels de filtros	Seleccionar filtros y mostrar resultados

6.2 Backend

Clase / interfaz	Responsabilidad
LoginController	Autenticar, crear principal/claims y cerrar sesión

DashboardController	Mostrar panel a usuarios autenticados
AncianosController	CRUD de internos y autorización por roles
FamiliaresController	CRUD de familiares y relación con pacientes
SolicitudesController	Gestionar solicitudes y coordinación con notificaciones
NotificacionesClient	Consumir el microservicio de notificaciones
AsiloDbContext	Acceso a entidades y SQL Server mediante EF Core
Servicios futuros	Encapsular reglas clínicas, financieras, reportes y auditoría

7. Prototipo

El prototipo funcional se construye directamente sobre ASP.NET Core MVC. No es únicamente una maqueta visual: varias pantallas ya están conectadas a SQL Server y aplican autenticación y autorización.

Pantalla / módulo	Estado	Qué demuestra
Login	Implementado	Validación de usuario/correo, contraseña y cookie de autenticación
Dashboard por rol	Implementado	Claims, roles y visibilidad de módulos
Acceso denegado	Implementado	Bloqueo de funciones no permitidas
Internos	Implementado	CRUD completo conectado a SQL Server
Familiares	En progreso	Relación Paciente 1:N Familiar y formulario de registro
Solicitudes médicas	Base disponible	Flujo inicial y microservicio de notificaciones
Ficha médica	Pendiente	Se implementará después de Familiares
Laboratorio	Pendiente	Resultados y costos de exámenes
Farmacia	Pendiente	Entrega de medicamentos y costos
Finanzas y reportes	Pendiente	Cobros, pagos, donaciones, gastos y reportes

7.1 Wireframe funcional del prototipo

Pantalla de Login

ASILO CABEZA DE ALGODÓN
Inicio de sesión
[Usuario o correo]
[Contraseña]
[INICIAR SESIÓN]
Mensajes de validación / acceso

Dashboard por rol

Bienvenido: usuario	Rol actual	Cerrar sesión
Internos	Familiares	Ficha médica
Solicitudes	Laboratorio	Farmacia / Reportes

Evidencias recomendadas para la presentación del prototipo:

- Login funcionando con un usuario Administrador.
- Dashboard mostrando los módulos del Administrador.
- Ingreso con Farmacia y demostración de Acceso denegado al módulo Internos.
- Registro, edición y eliminación de un interno.
- Verificación del registro directamente en SQL Server.

- Registro de un familiar asociado a un paciente.
- Prueba de la API de notificaciones y evidencia de comunicación entre servicios.

8. Estrategia de despliegue público

Para la entrega final se recomienda publicar el sistema en la nube en lugar de abrir puertos de la red doméstica. Esto simplifica el acceso público y reduce riesgos sobre otros dispositivos del hogar.

8. Construir imágenes Docker de Asilo.Web y Asilo.Notificaciones.Api.
9. Publicar las imágenes o construirlas directamente en la VPS/servicio cloud.
10. Configurar una instancia de SQL Server accesible únicamente desde la red privada del servidor.
11. Configurar variables de entorno para cadenas de conexión y credenciales del correo.
12. Colocar un proxy inverso con HTTPS delante de la aplicación.
13. Crear copias de seguridad automáticas de la base de datos.
14. Usar GitHub para versionar el código y registrar la versión entregada.

9. Riesgos, control y seguimiento

Riesgo	Impacto	Mitigación
Retraso en módulos clínicos	Alto	Priorizar ruta crítica: Familiares → Ficha → Solicitudes → Visitas → Lab/Farmacia
Cambios en requisitos	Medio	Matriz de trazabilidad y control de cambios
Pérdida de datos	Alto	Backups de SQL Server y repositorio Git
Contraseñas de prueba	Alto	Reemplazar texto plano por PasswordHasher/PBKDF2 antes de entrega final
Problemas de Docker/SQL	Medio	Usar SQL Server accesible para contenedores; no depender de LocalDB en Linux
Fallas de correo	Medio	Registrar estado de notificación y permitir reintento
Falta de tiempo para pruebas	Alto	Pruebas por módulo durante el desarrollo, no solo al final

10. Criterios de cierre de la fase de diseño

- Planificación actualizada con actividades, horas, responsables, dependencias e hitos.
- Arquitectura lógica y de despliegue documentada.
- Componentes y clases principales definidos.
- Patrones UX/UI y reglas de navegación establecidos.
- Clases de interfaz frontend/backend identificadas.
- Prototipo funcional presentado con evidencias.
- Riesgos principales registrados y mitigaciones definidas.
- Diseño alineado con ERS, UML, ERD y requisitos de seguridad.